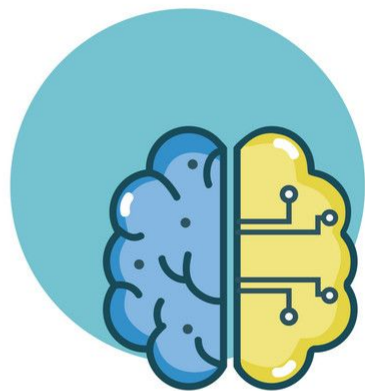


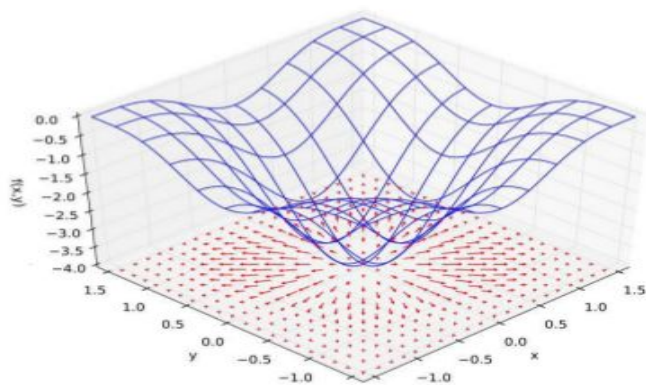
INTRODUCTION TO MACHINE LEARNING

NEURAL NETWORKS II



Elisa Ricci



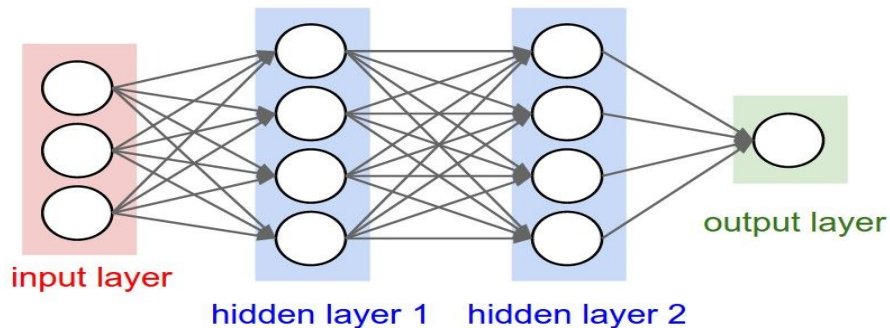


TRAINING A NEURAL NETWORK

FEEDFORWARD NETWORKS

The function f is a composition of multiple functions:

$$f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$$



FEEDFORWARD NETWORKS

- Goal: Approximate some unknown ideal function $f^* : \mathcal{X} \rightarrow \mathcal{Y}$
- Feedforward Network:
 - Define parametric mapping $f(\mathbf{x}_i; \Theta)$
 - Learn parameters to get a good approximation of f^* from available sample
- The computation can be described by a **Directed Acyclic Graph (DAG)**
 - Information flow in function evaluation begins at input, flows through intermediate computations to produce the output

MODELING CHOICES

- Need to choose:
 - Cost function
 - Form of output
 - Activation functions
 - Architecture (number of layers etc)
 - **Optimizer (for training)**

TRAINING A NEURAL NETWORK

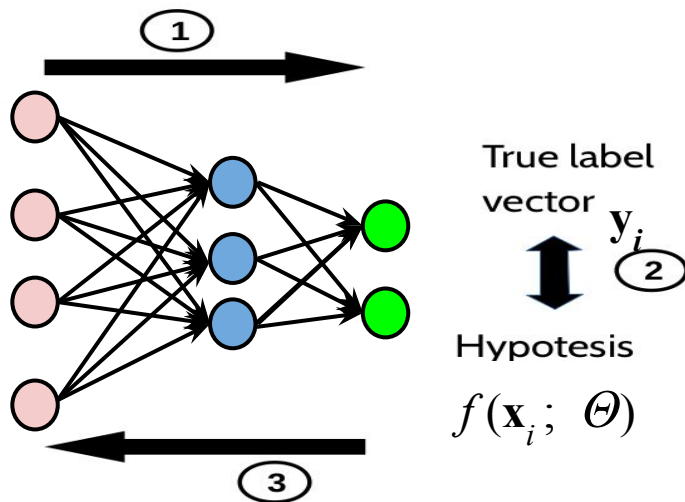
- **Learning = Optimization**
- Main idea: Given training samples $T = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_N, \mathbf{y}_N)\}$ adjust all the weights of the network Θ such that a cost function is minimized

$$\min_{\Theta} \sum_i L(\mathbf{y}_i, f(\mathbf{x}_i; \Theta))$$

- Choose your loss function (e.g. square loss, cross-entropy loss)
- Update the weights of each layer with **gradient descent**
- Use the **Backpropagation** to **compute the gradient efficiently**

SO FAR: BACKPROPAGATION

1. **Forward propagation:** sum inputs, produce activations, feed-forward
2. **Error estimation**
3. **Back propagate** the error signal and used it to update weights



GRADIENT DESCENT

Feedforward neural networks can be trained with Vanilla Gradient Descent

```
# Vanilla Gradient Descent

while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

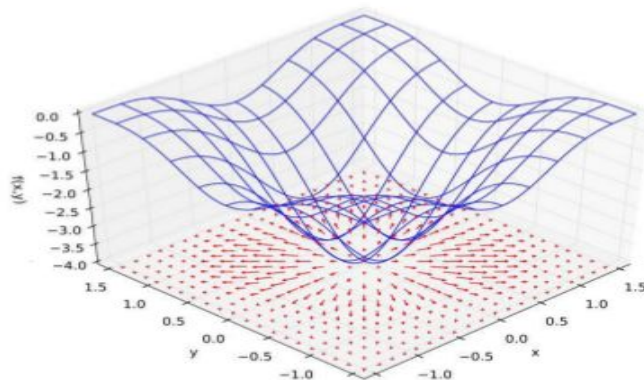
Gradient
descent
update rule

GRADIENT

- The gradient is the vector of partial derivatives wrt to all the coordinates of the weights:

$$\nabla_{\mathbf{w}} L = \left[\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \cdots \quad \frac{\partial L}{\partial w_M} \right]$$

- Each partial derivative measures how fast the loss changes in one direction.
- When the gradient is zero, i.e. all the partials derivatives are zero, the loss is not changing in any direction.
- Issues: local minima, saddle points



GRADIENT DESCENT

- **Gradient Descent** finds the set of parameters that makes the loss as small as possible
- The change of parameters depends on the gradients of the loss with respect to the network weights
- Backpropagation is a method for computing gradients
- What we see now: **Stochastic Gradient Descent (SGD)** and other optimization methods

BATCH GRADIENT DESCENT (BGD)

Input: Learning rate η_k , initial parameters $\mathbf{w}$

while stopping criteria not met **do**

 Compute gradient estimate over N examples

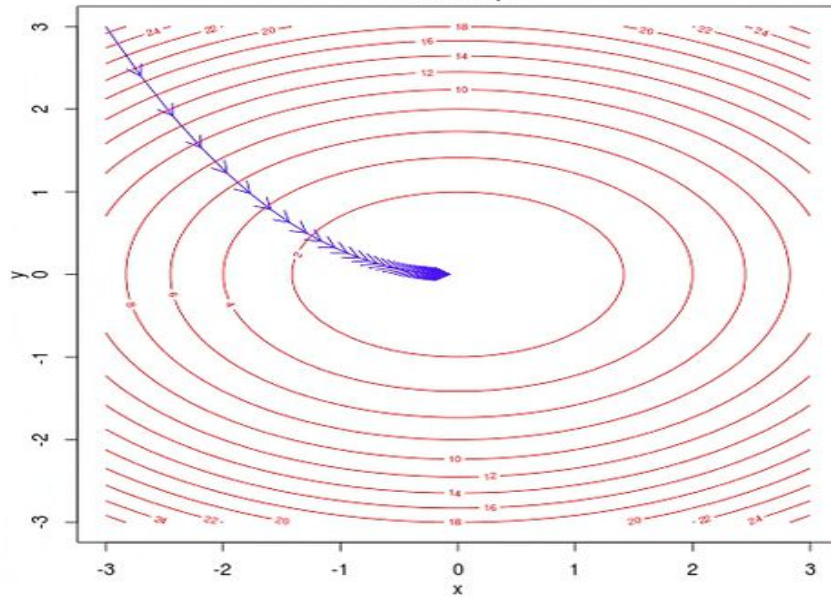
$$\mathbf{g} \leftarrow \frac{1}{N} \nabla_{\mathbf{w}} \sum_i L(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i)$$

 Apply update: $\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$

end while

The learning rate changes at each step, typically is decayed linearly.

BATCH GRADIENT DESCENT



- **Pros:** Gradient estimates are stable
- **Cons:** Need to compute gradients over the entire training for one update

STOCHASTIC GRADIENT DESCENT (SGD)

Input: Learning rate η_k , initial parameters $\mathbf{w}$

while stopping criteria not met **do**

 Sample one datapoint from training set $(\mathbf{x}_i, y_i)$

 Compute gradient estimate

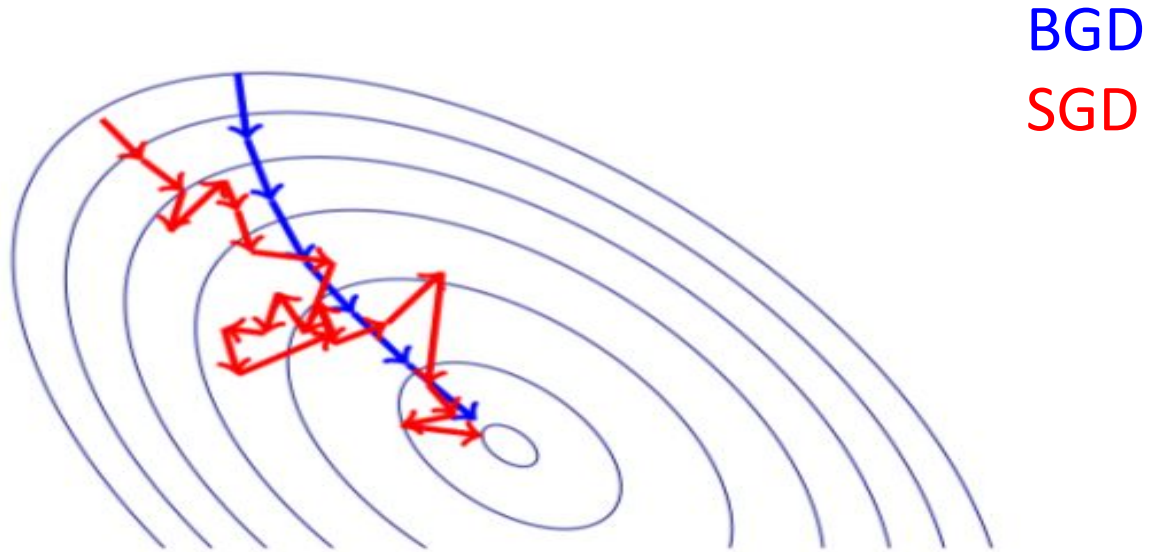
$$\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L(f(\mathbf{x}_i; \mathbf{w}), y_i)$$

 Apply update: $\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$

end while

The learning rate changes at each step, typically is decayed linearly.

BGD VS SGD



BGD VS SGD

BGD

$$\mathbf{g} \leftarrow \frac{1}{N} \nabla_{\mathbf{w}} \sum_i L(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i)$$
$$\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$$

SGD

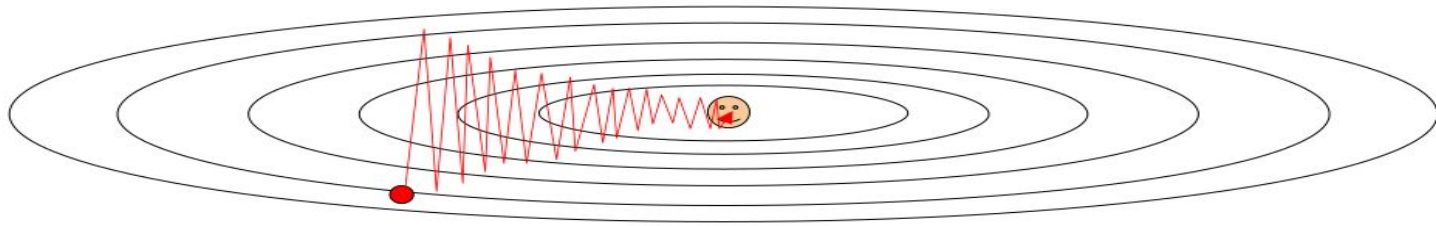
$$\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i)$$
$$\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$$

MINIBATCHES

- **Problem:** gradient estimates can be very noisy
- One obvious solution is to use **mini-batches** (small sets of samples)
- **Advantage:**
 - Computation time per update does not depend on number of training examples N
 - It permits computation on extremely large datasets
 - Often parallel implementation
 - Using GPUs it is common for power of 2 batch sizes to offer better runtime (some kinds of hardware achieve better runtime with specific sizes of arrays).

MOMENTUM

Problem with SGD: with some error surfaces very slow progress along flat direction, jitter along steep one!



MOMENTUM

Introduce a new variable **v**: the velocity

The velocity is an exponentially decaying moving average of the negative gradient

Input: Learning rate η_k , initial parameters **w**, initial velocity **v**, momentum parameter α

while stopping criteria not met **do**

 Sample one datapoint from training set $(\mathbf{x}_i, \mathbf{y}_i)$

 Compute gradient estimate

$$\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i)$$

 Compute velocity update

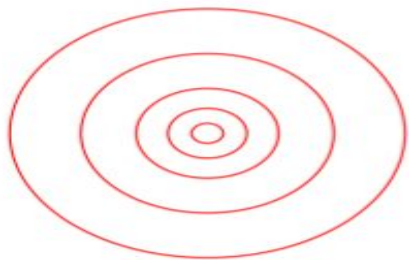
$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \eta_k \mathbf{g}$$

 Apply update: $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$

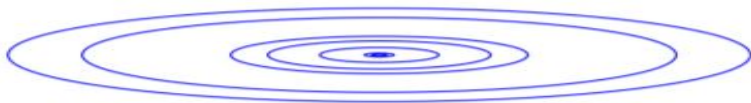
end while

ADAPTIVE LEARNING RATE METHODS

- So far we have assigned the same learning rate to all features
- If the features vary in importance and frequency, is this a good idea?
- The learning rate is one of the hyperparameters most difficult to set in neural networks

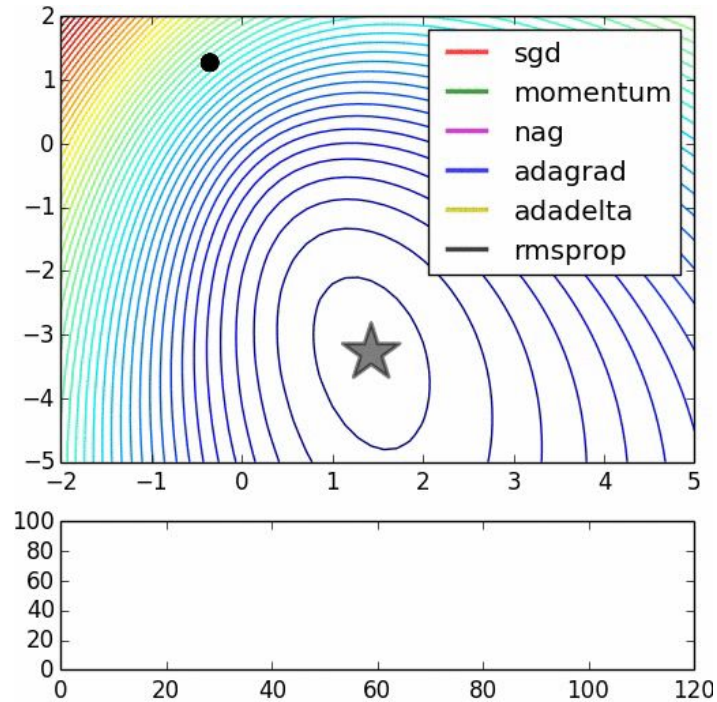


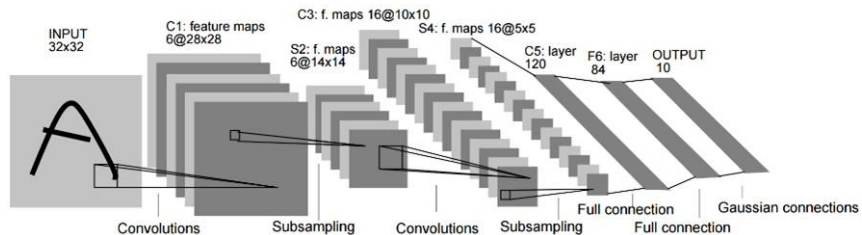
Easier: all the features important



Harder

DIFFERENT METHODS





CONVOLUTIONAL NEURAL NETWORKS

STRUCTURED DATA

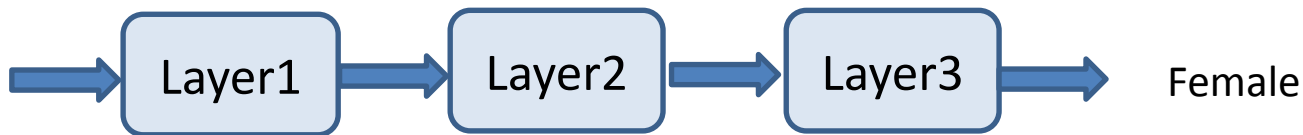
- Some applications naturally deal with an input space which is locally structured, i.e. **spatial** or temporal.
- **Images**, language, etc. vs arbitrary input features.
- Neural networks are extremely powerful in this case.



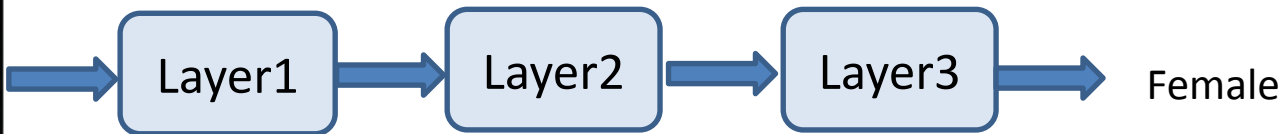
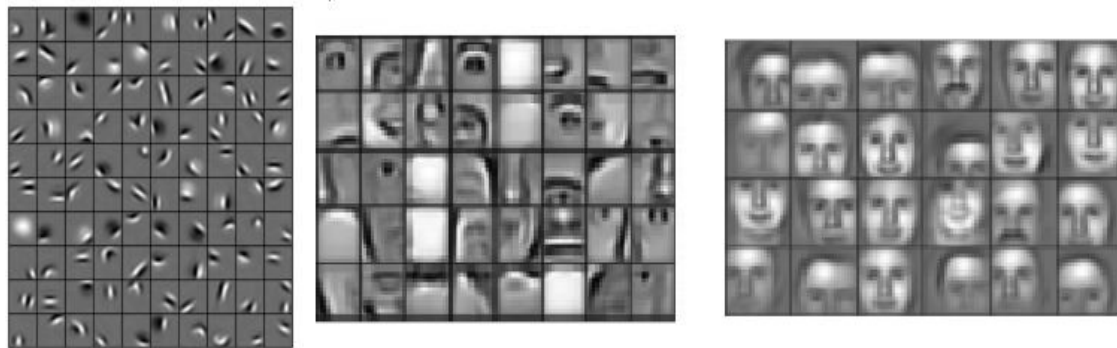
Tomorrow, and
tomorrow, and
tomorrow; creeps
in this petty pace
from day to day,
until the last syll-
able of recorded
time. And all our
yesterdays have
lighted fools the
way to dusty

FROM PIXELS TO LABELS

- Learn a hierarchy of features
- Each layer of hierarchy extracts features from output of previous layer
- Train all layers jointly



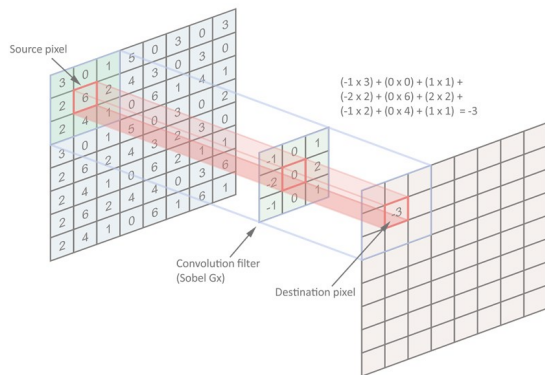
IN CONVOLUTIONAL NEURAL NETWORKS...



CONVOLUTIONAL NEURAL NETWORKS

Convolutional networks are simply neural networks that use **convolution** in place of general matrix multiplication **in at least one of their layers**.

What is a convolution?



$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n).$$

RECAP: CONVOLUTION

- Convolution is a general purpose filter operation for images.
- A kernel matrix is applied to an image.
- It works by determining the value of a central pixel by adding the weighted values of all its neighbors together.
- The output is a new modified filtered image.

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n).$$

- Can be used to smooth, sharpen, enhance...
- It is a commutative operation.

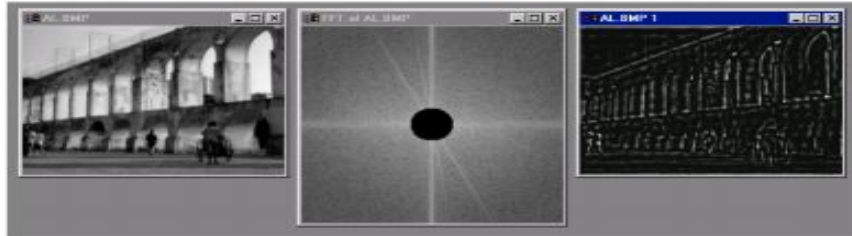
RECAP: CONVOLUTION

Sharply peaked kernel K for edge detection

$I(i, j)$

$K(i, j)$

$S(i, j)$



Kernels K_1 - K_4 for line detection

<table><tr><td>-1</td><td>-1</td><td>-1</td></tr><tr><td>2</td><td>2</td><td>2</td></tr><tr><td>-1</td><td>-1</td><td>-1</td></tr></table> Horizontal lines	-1	-1	-1	2	2	2	-1	-1	-1	<table><tr><td>-1</td><td>2</td><td>-1</td></tr><tr><td>-1</td><td>2</td><td>-1</td></tr><tr><td>-1</td><td>2</td><td>-1</td></tr></table> Vertical lines	-1	2	-1	-1	2	-1	-1	2	-1
-1	-1	-1																	
2	2	2																	
-1	-1	-1																	
-1	2	-1																	
-1	2	-1																	
-1	2	-1																	
<table><tr><td>-1</td><td>-1</td><td>2</td></tr><tr><td>-1</td><td>2</td><td>-1</td></tr><tr><td>2</td><td>-1</td><td>-1</td></tr></table> 45 degree lines	-1	-1	2	-1	2	-1	2	-1	-1	<table><tr><td>2</td><td>-1</td><td>-1</td></tr><tr><td>-1</td><td>2</td><td>-1</td></tr><tr><td>-1</td><td>-1</td><td>2</td></tr></table> 135 degree lines	2	-1	-1	-1	2	-1	-1	-1	2
-1	-1	2																	
-1	2	-1																	
2	-1	-1																	
2	-1	-1																	
-1	2	-1																	
-1	-1	2																	

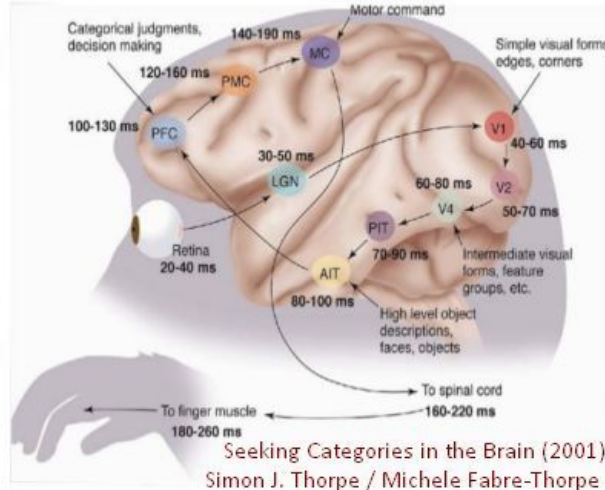
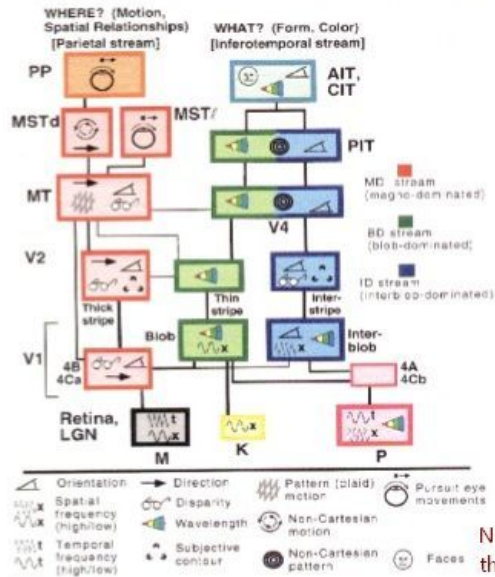
RECAP: CONVOLUTION

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

CONVOLUTIONAL NEURAL NETWORKS

Inspired by mammalian visual cortex.



Neural mechanisms of form and motion processing in the primate visual system [Van Essen & Gallant, 1994]

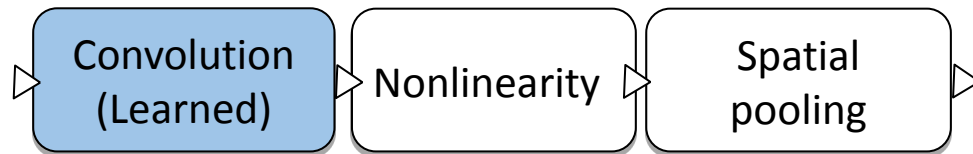
VISUAL CORTEX

- The visual cortex contains a complex arrangement of cells, which are sensitive to small sub-regions of the visual field, called a **receptive field**.
- These cells act as local filters over the input space and are well-suited to exploit the strong spatially local correlation present in natural images.
- Two basic cell types:
 - **Simple cells** respond maximally to specific edge-like patterns within their receptive field.
 - **Complex cells** have larger receptive fields and are locally invariant to the exact position of the pattern.

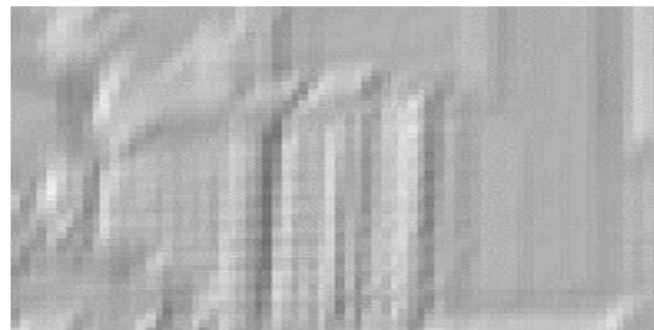
CNN: ARCHITECTURE

- Feedforward neural network with specialized connectivity structure
- Typically CNN layers transform the input matrix into an output class prediction.
- There are a few distinct types of operations:
 - Convolution
 - Non-linearity
 - Pooling

CNN MOTIF



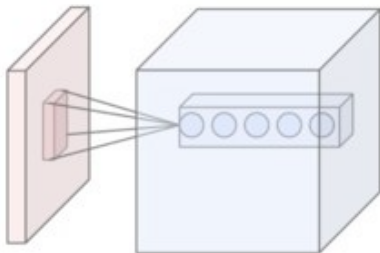
Input



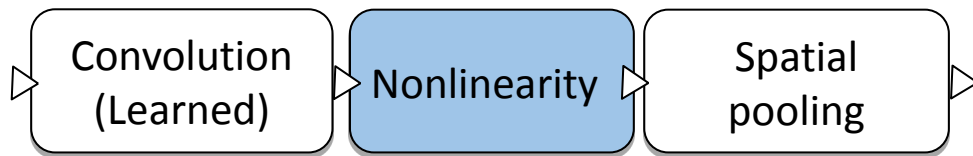
Feature Activation Map

CONVOLUTION

- **Convolutional layer:** core layer of CNNs.
- Consists of a set of **learned filters**.
- Each filter covers a spatially small portion of the input data (**receptive field**).
- Each filter is convolved across the dimensions of the input data, producing a **multi-dimensional feature map**.
- **Intuition:** the network will learn filters that activate when they see some specific type of feature at some spatial position in the input.



CNN: ARCHITECTURE

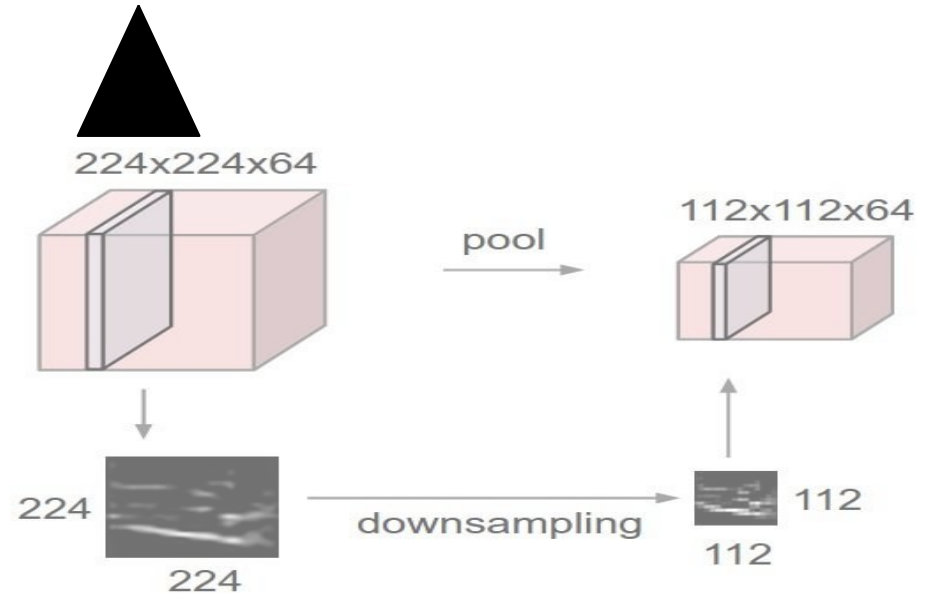
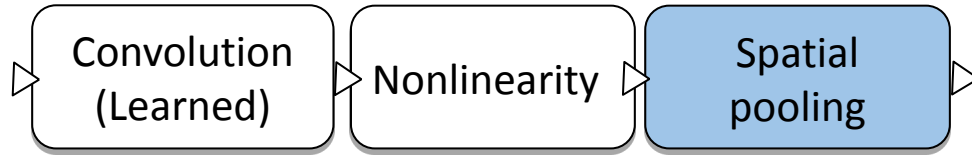


Apply elementwise

$$h(x) = \max(0, x)$$

Increase the nonlinearity of the entire architecture without affecting the receptive fields of the convolution layer.

CNN: ARCHITECTURE

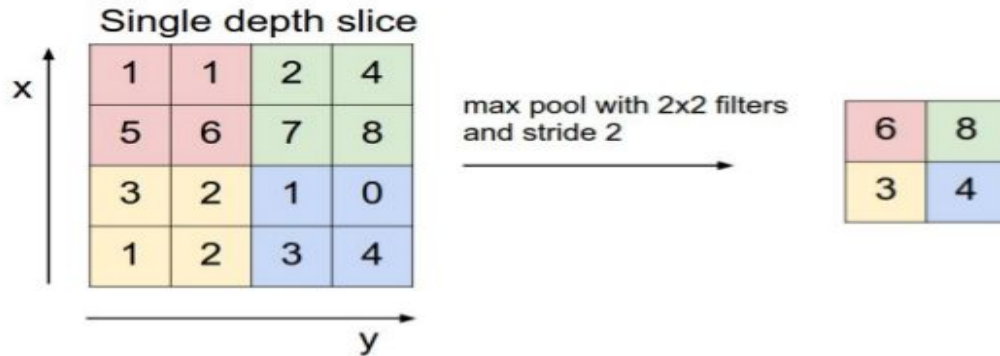


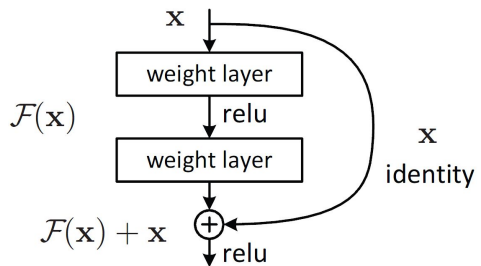
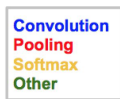
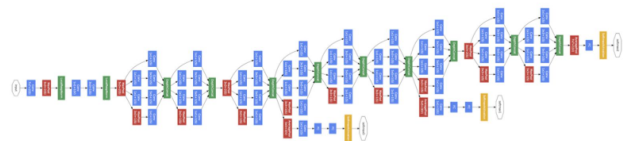
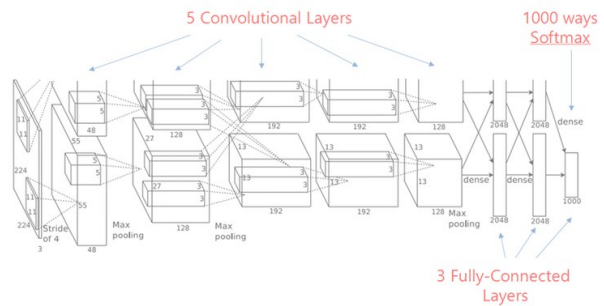
Pooling: to provide invariance to translations

POOLING

By progressively reducing the spatial size of the representation we **reduce the amount of parameters** and computation in the network and also **control overfitting**.

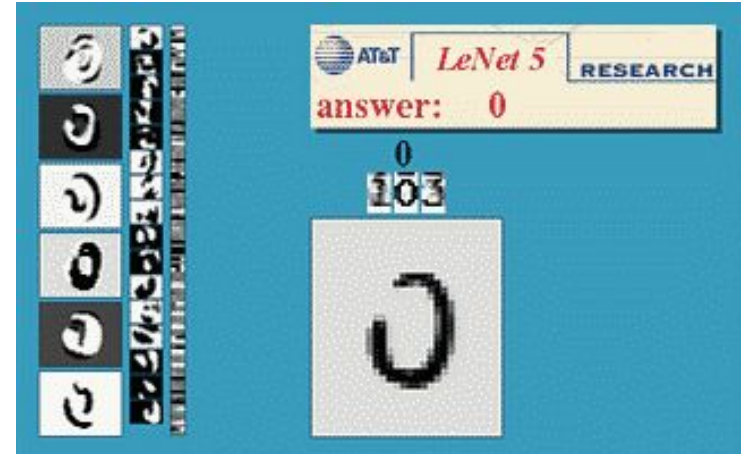
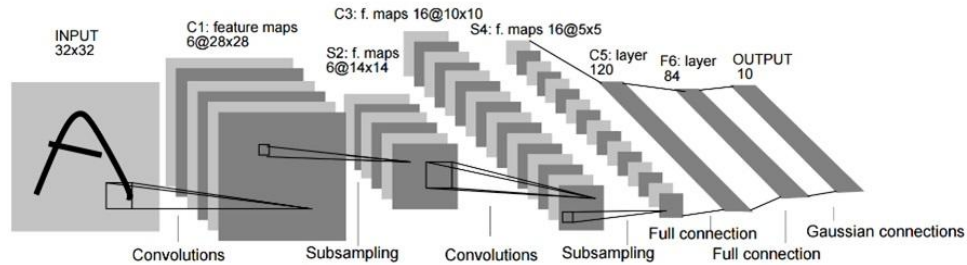
Example: max pooling





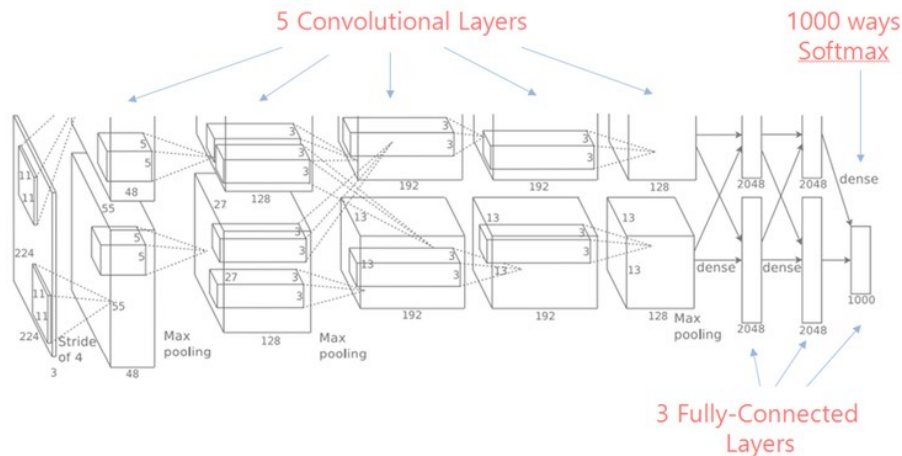
CONVOLUTIONAL NEURAL NETWORKS ARCHITECTURES

LeNet - 1998



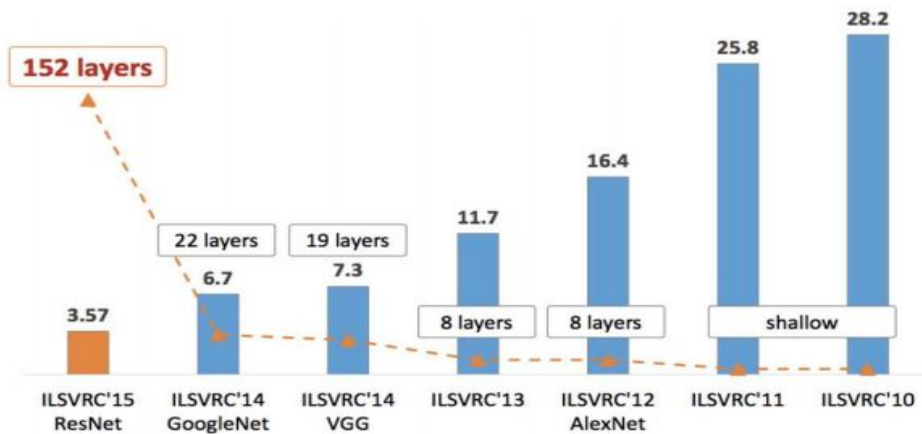
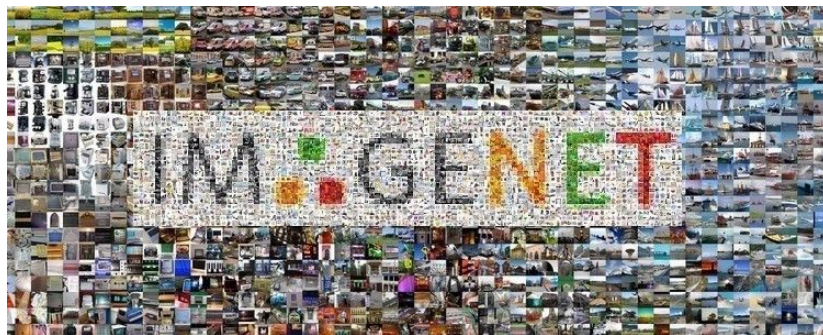
ALEXNET - 2012

- Similar framework to LeNet but...
- Bigger model (7 hidden layers, 650K units, 60M params)
- More data (10^6 vs. 10^3 images)
- GPU implementation (50x speedup over CPU) - Trained on two GPUs for a week



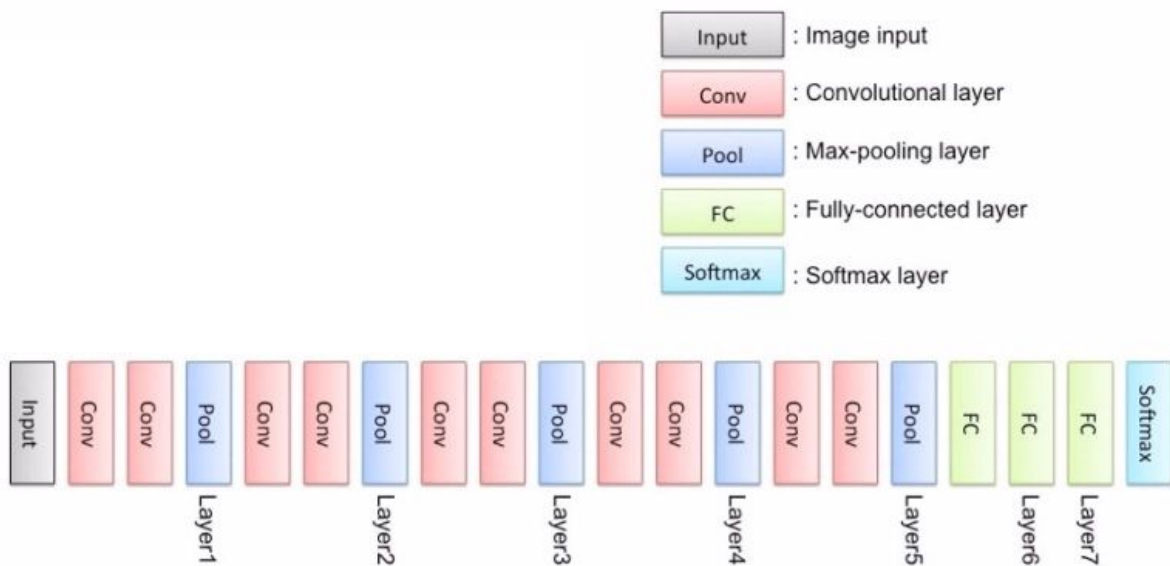
GOING DEEPER

Classification: ImageNet Challenge top-5 error



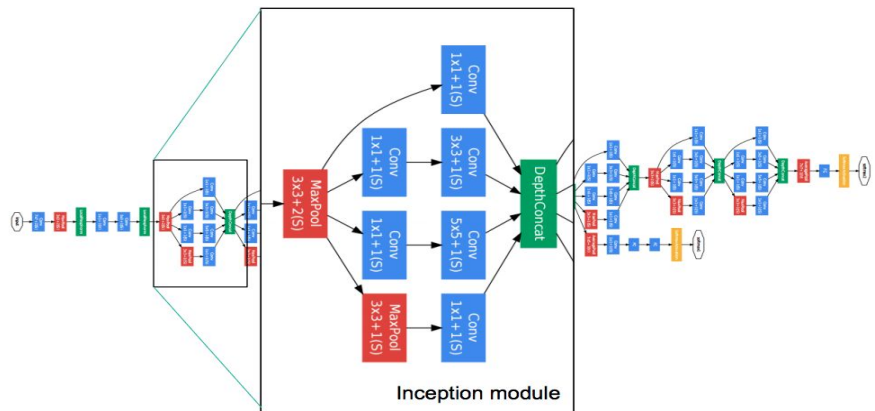
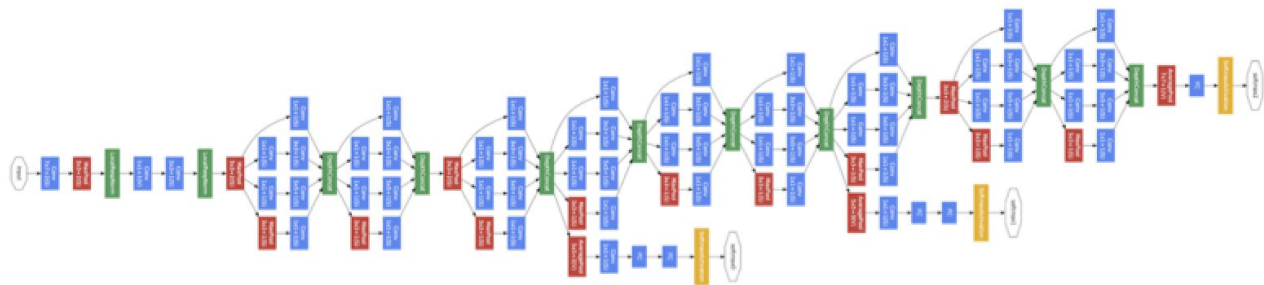
VGG - 2014

Similar motif to AlexNet



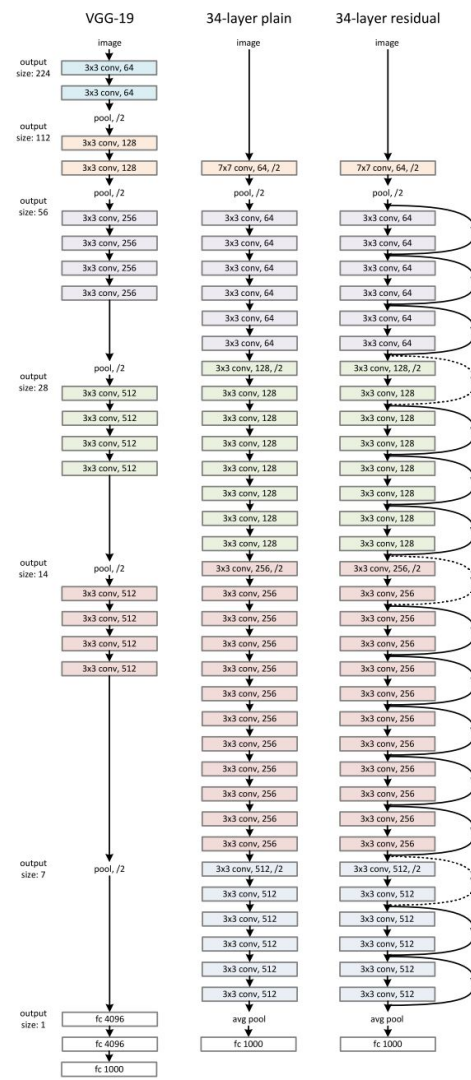
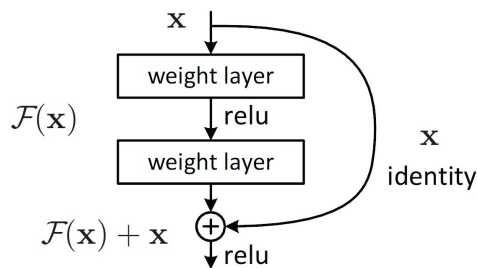
GOOGLNET

- Has 12x fewer parameters than AlexNet
- Gets rid of fully connected layers
- Inception Module



RESNET

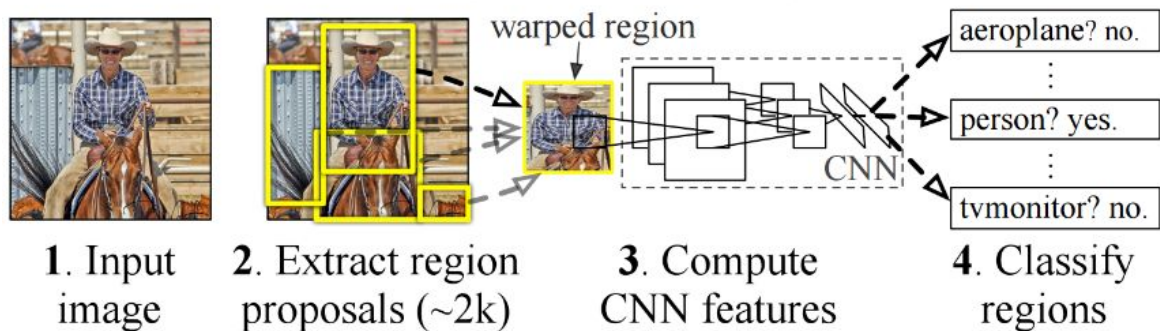
- Residual Block: improved performance of very deep nets
- Solve the **degradation problem** enabling the deeper layers to propagate the information from the shallow layers directly with identity mapping.
- Introduces batch normalization to improve training



BEYOND CLASSIFICATION

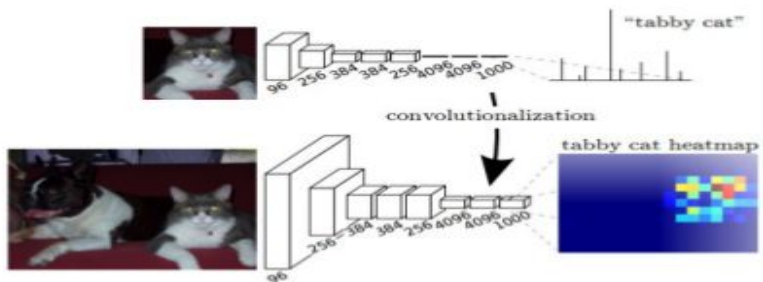
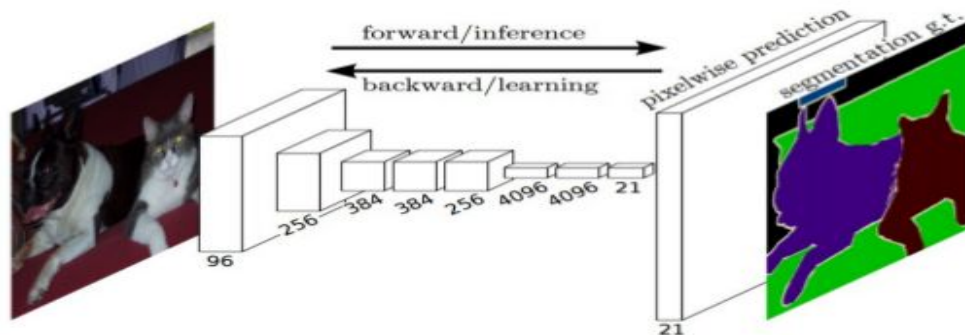
Detection

- First approach: R-CNN (Regions with CNN features)
- Trained on ImageNet classification
- Fine-tune CNN on PASCAL-VOC
- Nowadays more sophisticated methods exist



BEYOND CLASSIFICATION

- Semantic Segmentation



BEYOND CLASSIFICATION

- Structured Regression



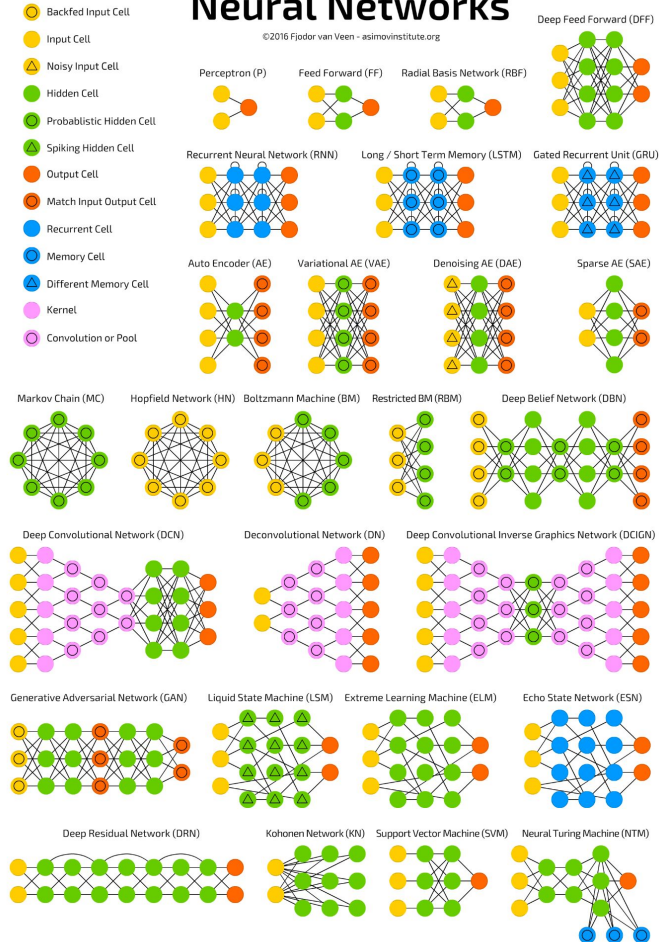
CNN: SUMMARY

- In a feedforward neural network, units are organized into layers and the units at a given layer only get input from units in the layer below.
- CNNs are feedforward networks. However, unlike standard vanilla feedforward networks, units in a CNN have a **spatial arrangement**.
- At each layer, units are organized into 2D grids, the **feature maps**.
- Each feature map is the result of a convolution. The same **convolutional filter** is applied at each location. The weights are different across feature maps.
- A unit at a particular location on the 2D grid can only receive input from units at a similar location at the layer below.
- Need labeled data: supervised learning model
- Flexible to many applications

A mostly complete chart of

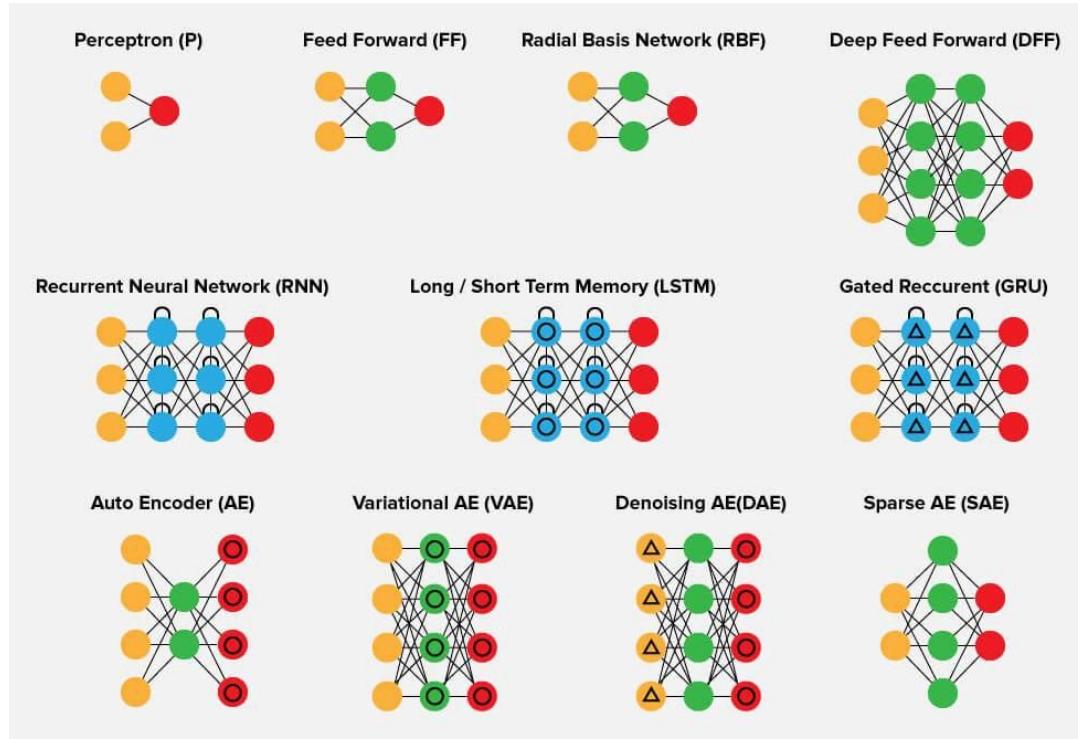
Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org



OTHER
NEURAL NETWORKS

MANY MODELS FOR DIFFERENT NEEDS

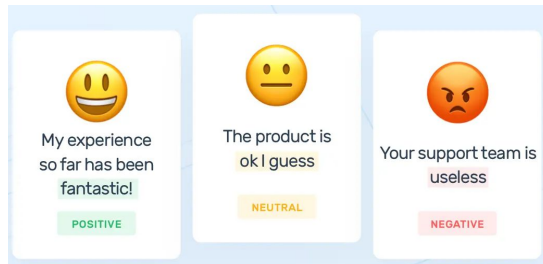


SEQUENTIAL PREDICTION TASKS

- So far, we focused mainly prediction problems with **fixed-size** inputs and outputs
- We discussed the flexibility of CNN to address a wide range of tasks.
- But what if the input and/or output is a **variable-length sequence**?
- Many applications where we need this...



Document classification

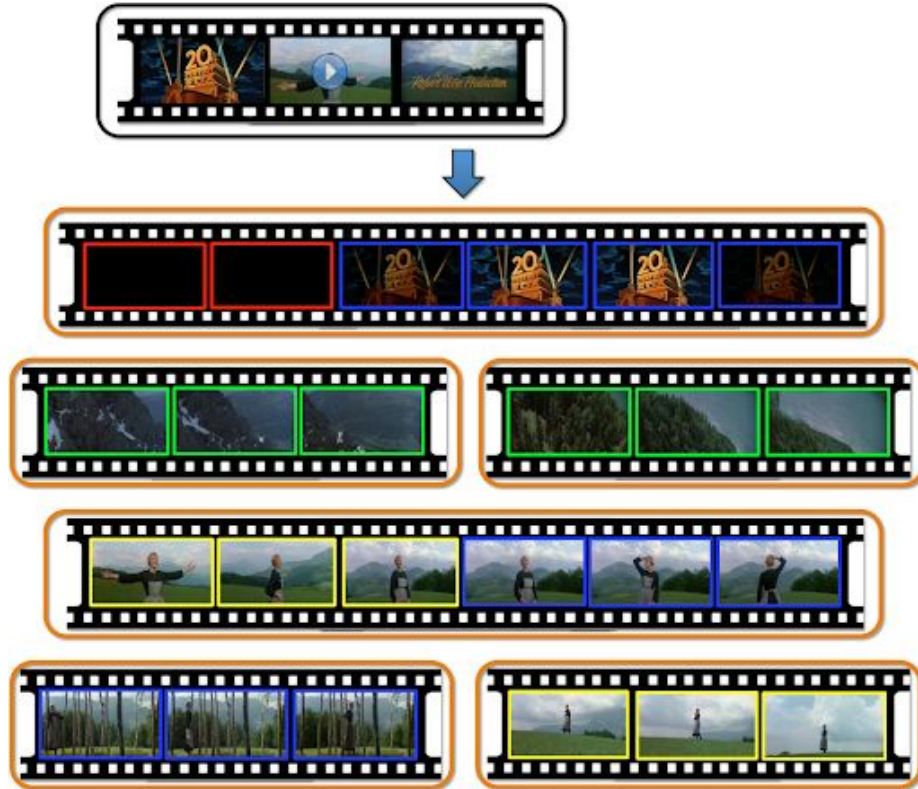


Sentiment Analysis



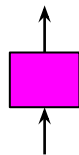
Image Captioning

EXAMPLE: VIDEO FRAME PREDICTION



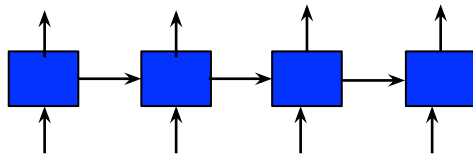
WHAT IS NEW?

Single2Single



Feedforward Network

Multiple2Multiple

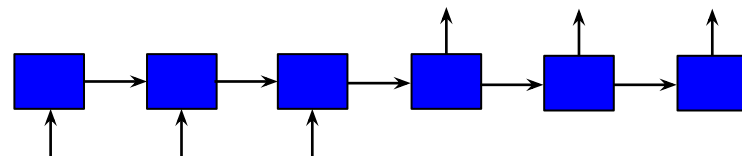
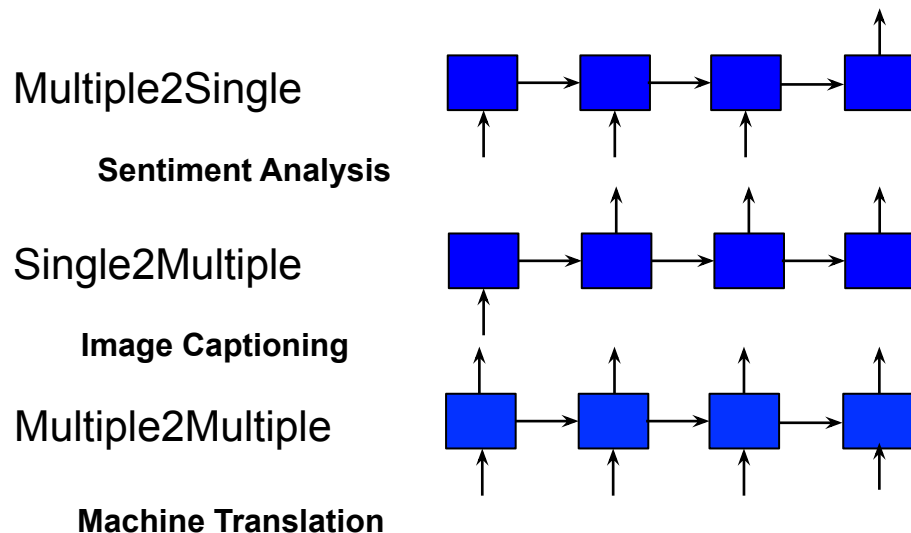


Recurrent Network

Video Frame Prediction

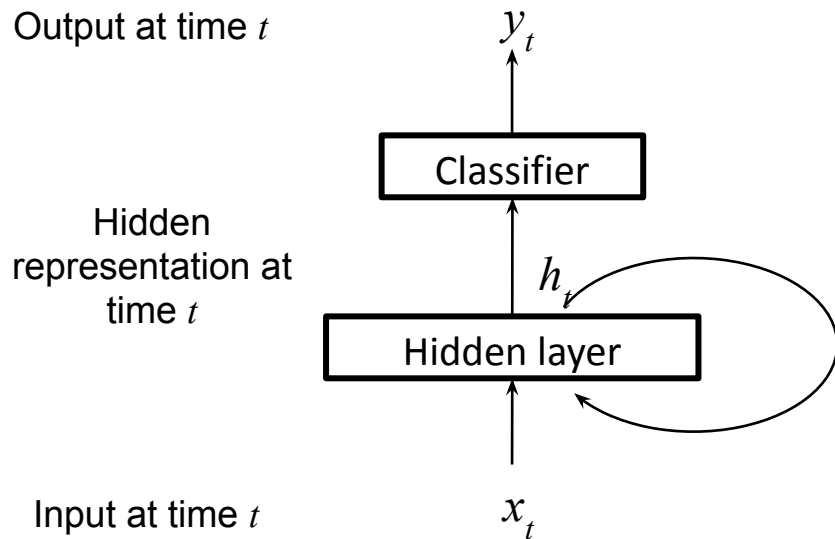
RECURRENT NEURAL NETWORK (RNN)

RNN can address a wide range of tasks



RECURRENT NEURAL NETWORK (RNN)

- Introduces cycles, recurrences

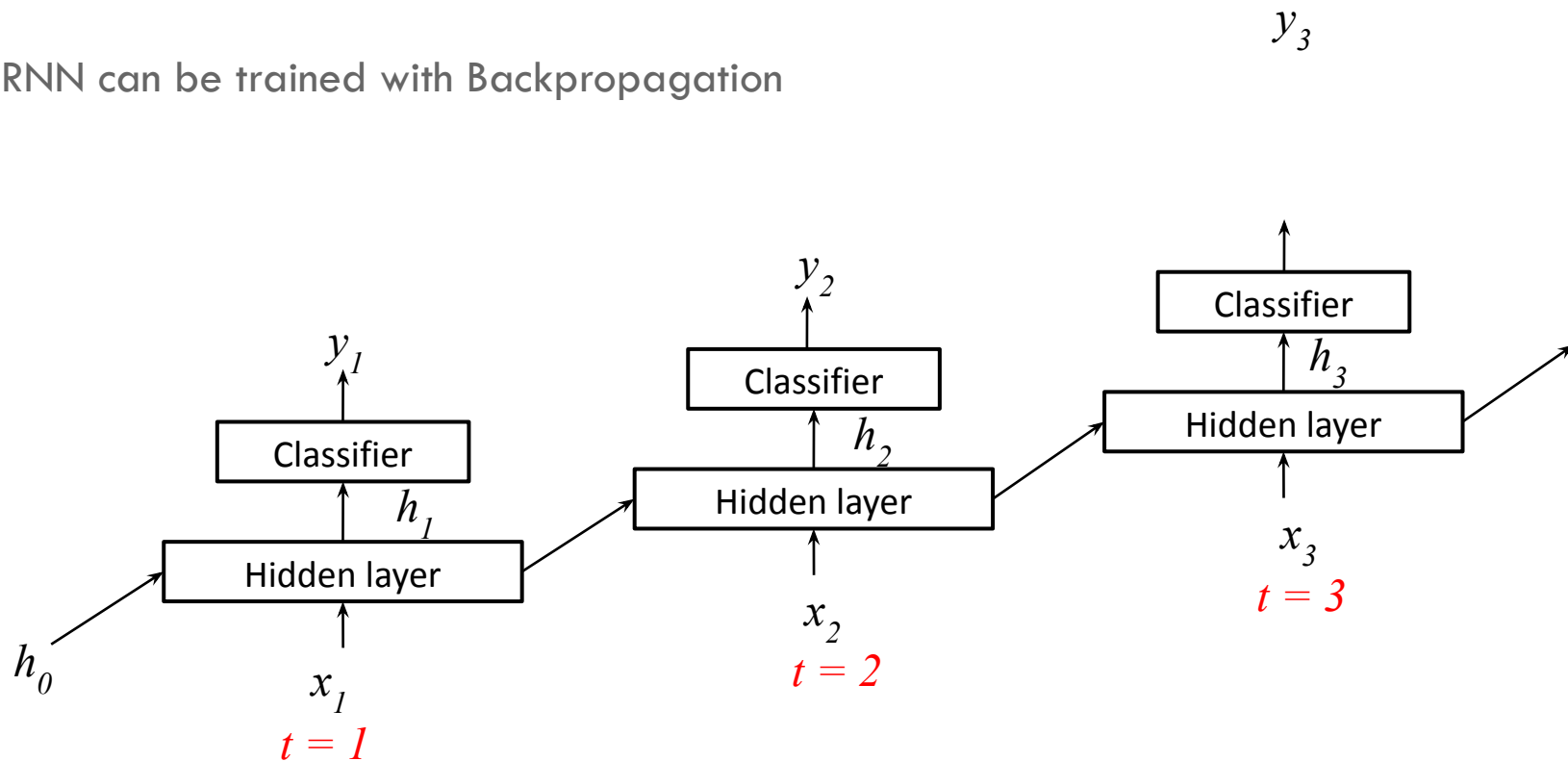


$$h_t = f_W(x_t, h_{t-1})$$

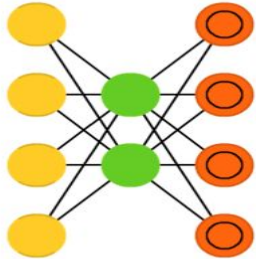
new state function of W input at time t old state

RECURRENT NEURAL NETWORK (RNN)

RNN can be trained with Backpropagation



UNSUPERVISED LEARNING: AUTOENCODERS



Autoencoders (AE) are somewhat similar to FFNNs as AEs are more like a different use of FFNNs than a fundamentally different architecture. The basic idea behind autoencoders is to encode information (as in compress, not encrypt) automatically, hence the name. The entire network always resembles an hourglass like shape, with smaller hidden layers than the input and output layers. AEs are also always symmetrical around the middle layer(s) (one or two depending on an even or odd amount of layers). The smallest layer(s) is/are almost always in the middle, the place where the information is most compressed (the chokepoint of the network). Everything up to the middle is called the encoding part, everything after the middle the decoding and the middle (surprise) the code. One can train them using backpropagation by feeding input and setting the error to be the difference between the input and what came out. AEs can be built symmetrically when it comes to weights as well, so the encoding weights are the same as the decoding weights.

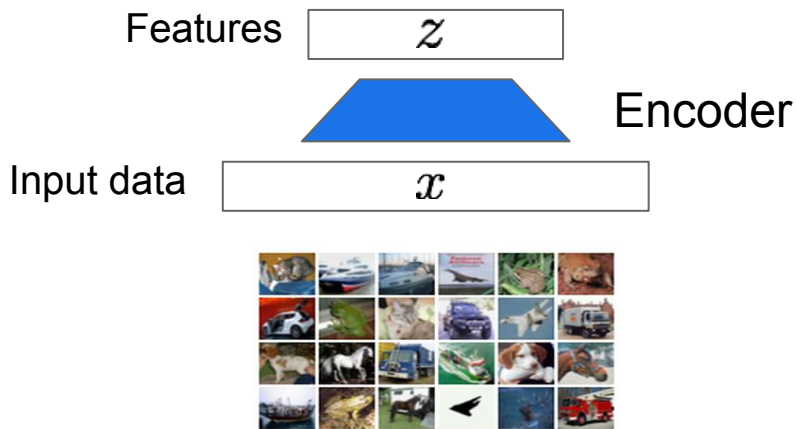
Bourlard, Hervé, and Yves Kamp. "Auto-association by multilayer perceptrons and singular value decomposition." Biological cybernetics 59.4-5 (1988): 291-294.

[Original Paper PDF](#)

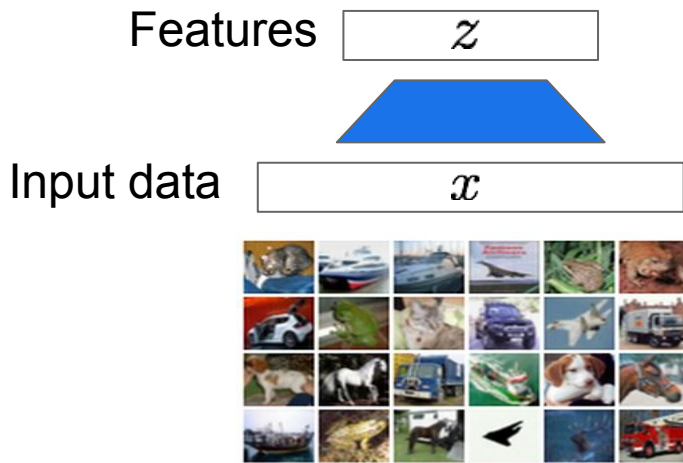
<https://www.asimovinstitute.org/neural-network-zoo/>

AUTOENCODERS: DIMENSIONALITY REDUCTION

- Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data
- Features should capture meaningful factors of variation in data: z usually smaller than x



AUTOENCODERS



Encoder

Originally: Linear + nonlinearity (sigmoid)

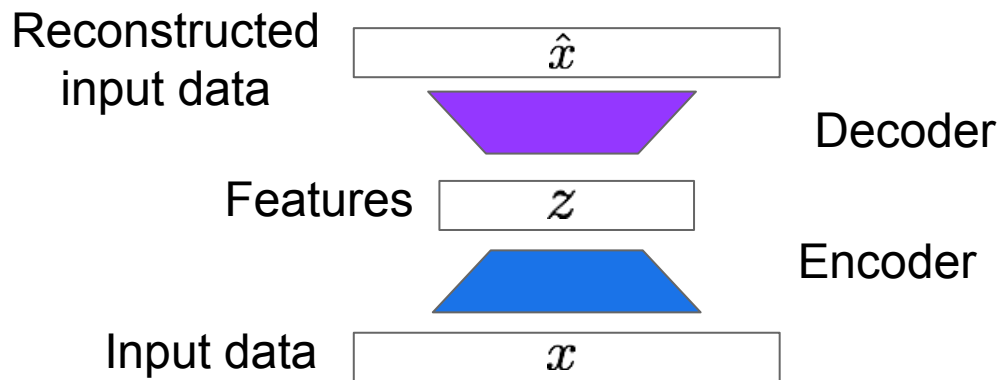
Later: Deep, fully-connected

Later: CNN with ReLU

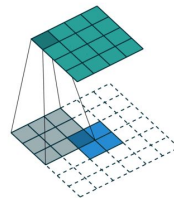
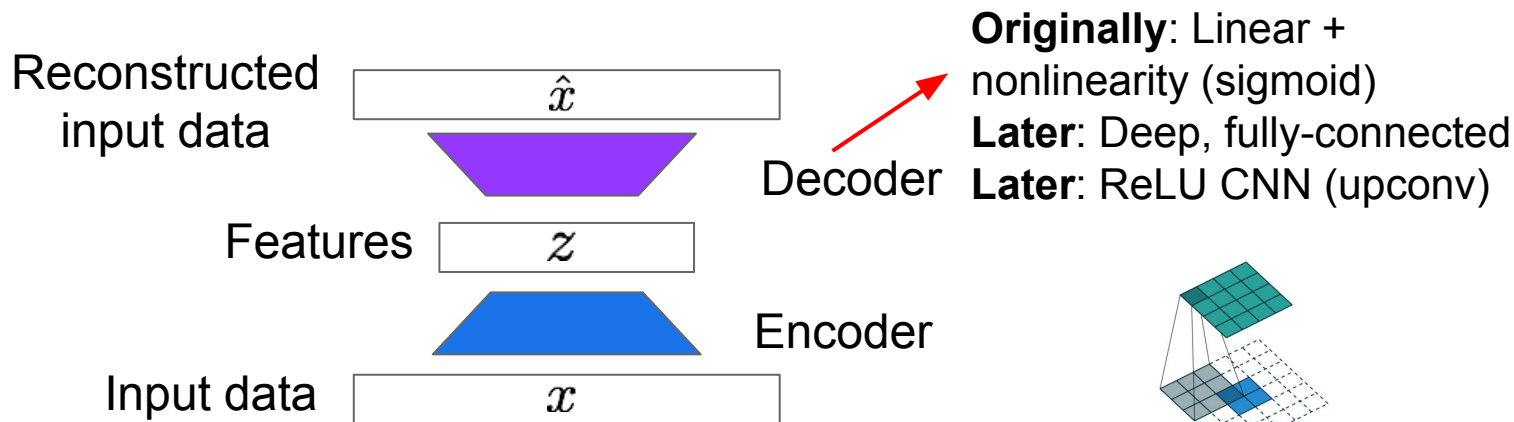
AUTOENCODERS

How to learn this feature representation?

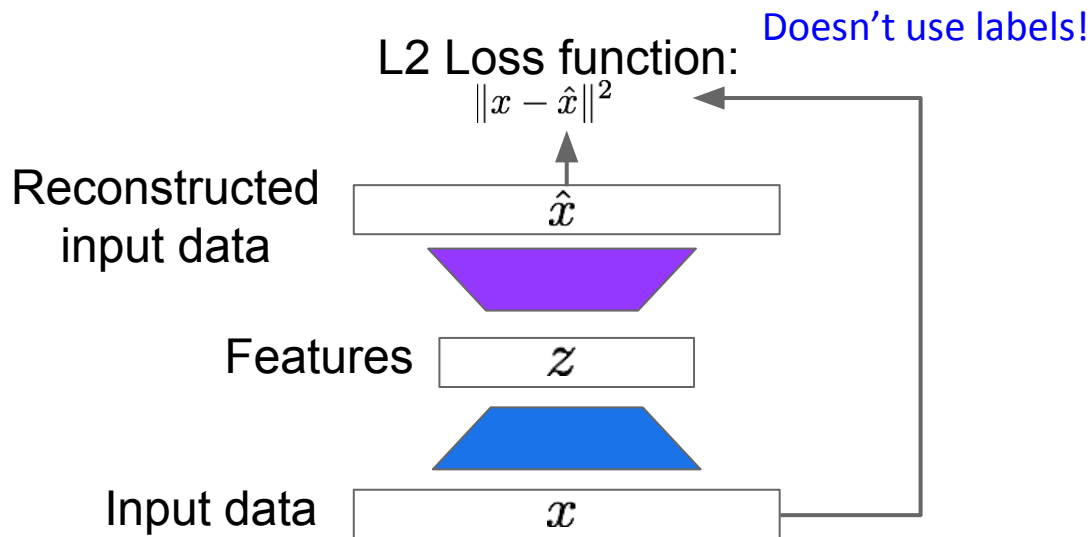
- Train such that features can be used to reconstruct original data
- **“Autoencoding” - encoding itself**



AUTOENCODERS

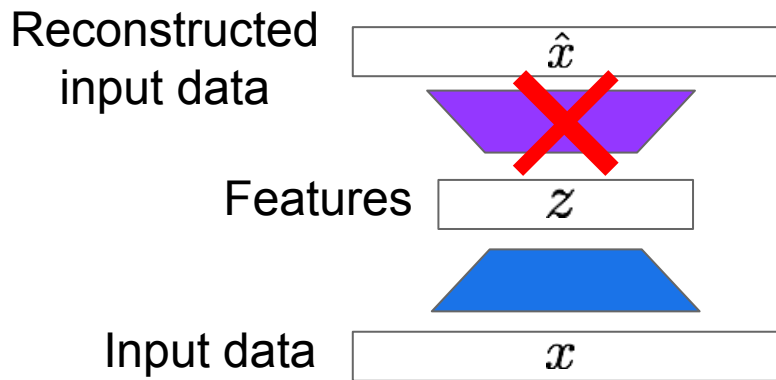


AUTOENCODERS



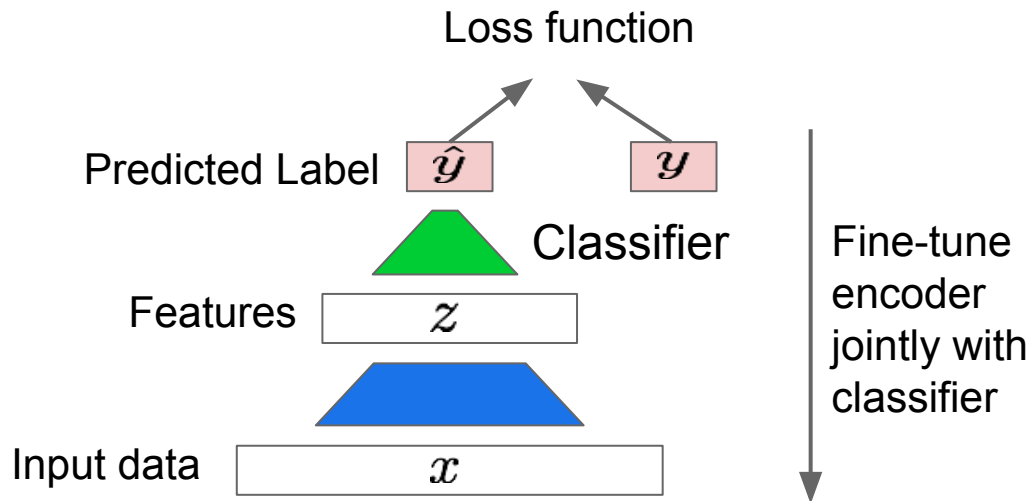
AUTOENCODERS

- After training, we throw away decoder
- Encoder can be used to initialize a supervised model



AUTOENCODERS

- After training, we throw away decoder
- Encoder can be used to initialize a supervised model



QUESTIONS?

